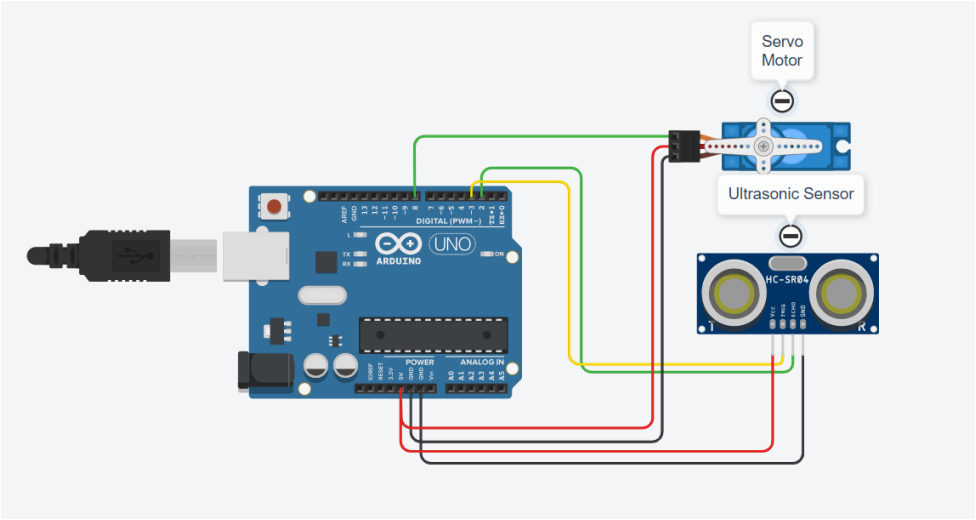

Figure 1: Ultrasonic with Servo Motor

Expt No:
03
Date:
/ /2025

CLOSED-LOOP CONTROL SYSTEM IMPLEMENTATION

1. AIM:

To design and implement a closed-loop feedback control system that uses an ultrasonic sensor to measure the distance to an object and controls a servo motor's angle to maintain a predefined distance (setpoint).

2. APPARATUS REQUIRED:

a) Microcontroller: Arduino UNO R3,
b) Sensor: Ultrasonic Distance Sensor (HC-SR04),
c) Actuator: Standard Servo Motor (SG90),
d) Software: Arduino IDE,
e) Components: Jumper Wires, USB-B Cable, Breadboard,
f) Miscellaneous: Object to be tracked (e.g., a small box).

3. THEORY:

A closed-loop (or feedback) control system continuously compares the system's output (Process Variable) to a desired input (Setpoint). The difference, known as the error, is fed back to a controller, which then adjusts the system's input to minimize the error.

In this experiment:

a) Process Variable (PV): The distance measured by the ultrasonic sensor.
b) Setpoint (SP): The desired target distance (e.g., 20 cm).
c) Error: Setpoint - Measured Distance.
d) Controller: The Arduino, which calculates the error.
e) Actuator: The servo motor, which is moved to a new angle based on the error.

We will implement a simple Proportional (P) Controller, where the corrective action (servo angle adjustment) is directly proportional to the error: Output = Kp * Error. Kp is the "Proportional Gain," a tuning constant that determines how aggressively the system reacts.

4. PROCEDURE:

a) Connections:
   i. HC-SR04 VCC → Arduino 5V,
   ii. HC-SR04 GND → Arduino GND,
   iii. HC-SR04 Trig → Arduino Digital Pin 9,
   iv. HC-SR04 Echo → Arduino Digital Pin 10,
   v. Servo VCC → Arduino 5V,
   vi. Servo GND → Arduino GND,
   vii. Servo Signal → Arduino PWM Pin 11.

b) Code: Upload the following sketch to the Arduino.

```
// --- Ultrasonic Sensor with Servo Motor ---
#include <Servo.h>

const int trigPin = 9;
const int echoPin = 10;
const int servoPin = 11;

float setpoint = 20.0;     // Target distance in cm
float Kp = 2.0;            // Proportional Gain (Tune this value)

Servo myServo;
float currentAngle = 90;   // Initial servo position

void setup() {
  Serial.begin(9600);

  pinMode(trigPin, OUTPUT);
  pinMode(echoPin, INPUT);

  myServo.attach(servoPin);
  myServo.write(currentAngle); // Start at 90 degrees
  delay(1000);
}

void loop() {
  // 1. SENSE: Measure the Process Variable (Distance)
  float distance = readDistance();

  // 2. COMPARE: Calculate the error
  float error = setpoint - distance;

  // 3. CONTROL: Calculate output using P-controller (proportional to error)
  float output = Kp * error;

  // 4. ACTUATE: Update the servo angle and constrain it to valid range
  currentAngle = currentAngle + output;
  currentAngle = constrain(currentAngle, 0, 180);  // Ensure servo stays within physical limits

  myServo.write(currentAngle);

  // Print diagnostics
  Serial.print("Dist: "); Serial.print(distance);
  Serial.print(" cm, Err: "); Serial.print(error);
  Serial.print(", Servo Angle: "); Serial.println(currentAngle);

  delay(100); // Loop delay for stability
}

// Function to read distance from HC-SR04
float readDistance() {
  long duration;

  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);
  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);

  duration = pulseIn(echoPin, HIGH);

  // Calculate distance in cm
  return (duration * 0.034 / 2.0);
}
```

c) Observation: Open the Serial Monitor. Place an object in front of the sensor. Observe the Dist. and Servo Angle values. When the object is moved closer than the 20cm setpoint, the error becomes positive, and the servo angle increases (moves away). When the object is moved farther, the error becomes negative, and the servo angle decreases (moves closer) to track the object.

5. RESULT AND OBSERVATION:

The closed-loop control system was successfully implemented. The experiment demonstrated the fundamental principle of feedback, where sensor data (distance) was used to proportionally control an actuator (servo) to minimize system error and maintain a desired setpoint.